We are going to design an algorithm for course admission that considers:

1. Multiple courses, each with its own class size and compulsory subjects.
2. Total number of subjects for all courses is 5 (meaning each applicant has 5 subjects?).
3. Disabled applicants who meet all the compulsory requirements for a course get first preference.
4. After allocating for disabled, a quota for females: 50% of the remaining places are reserved for females.
5. If the number of females does not fill their quota, the remaining places are given to males who failed to get places in the 50% male quota (and vice versa?).
6. Points are calculated based on the sitting (year) and grades (A, B, C) with different point scales for each sitting.

Steps:

We assume we have:

- List of courses, each with:
    - course_id
    - class_size
    - compulsory_subjects (list of subjects that are required)
- List of applicants, each with:
    - applicant_id
    - disability_status (boolean)
    - gender (male/female)
    - sittings: list of sitting records, each record has:
        - year (2020, 2021, ...)
        - subjects: list of subject-grade pairs (subject, grade) for 5 subjects.

How points are calculated?

- For each subject, the grade is converted to a point based on the sitting year.
- The point conversion table for sittings (from the problem):

First sitting (lowest year, e.g., 2020): A=1, B=2, C=3
Second sitting (e.g., 2021): A=2, B=3, C=4
Third sitting (e.g., 2022): A=3, B=4, C=5
Fourth sitting (e.g., 2023): A=4, B=5, C=6
Fifth sitting (e.g., 2024): A=5, B=6, C=7

- We assume that an applicant can have multiple sittings, but the problem doesn't specify how to combine them.
- Since the problem says "First sitting is characterised by the lowest year", we assume we take the earliest sitting for each subject?
  But note: an applicant may have taken different subjects in different sittings.

However, the problem does not specify how to handle multiple sittings. We have to make an assumption.

Let's assume:
- An applicant can have multiple sitting records (each record for a year, with 5 subjects).
- We are going to consider the best grade for each subject? Or the most recent? Or the earliest?

Since the problem says "First sitting is characterised by the lowest year", we might take the earliest sitting for each subject?
But note: the points for the same grade increase with the sitting year, so later sittings are more valuable.

Alternatively, the problem might intend that we use the sitting with the lowest year (the first sitting) for the entire set of subjects?
But then what if an applicant has multiple sittings? The problem doesn't specify.

Let's re-read: "First sitting which would be equal to one academic year eg 2020 has the following points...".
This suggests that each sitting is a full set of 5 subjects in one year.

We assume that an applicant can have at most 5 sittings (one per year from 2020 to 2024) and we are to use the best sitting?
Or the most recent? The problem does not specify.

Since the problem says "First sitting is characterised by the lowest year", we might take the earliest sitting that the applicant has.

However, note that the points for the same grade are higher in later sittings, so there is an incentive to use later sittings.

We need clarification, but since it's not provided, let's assume we are to use the sitting with the lowest year (the first sitting) for which the applicant has all the required subjects?
But the problem says "the total number for subjects for all courses is 5", so we assume each sitting has 5 subjects.

Alternatively, we might calculate the total points for each sitting and then take the best sitting? But the problem doesn't say.

Since the problem is ambiguous, we will assume that we are to use the earliest sitting (the one with the lowest year) for which the applicant has the required subjects for the course.

However, note: the problem says "If someone is disabled and they have all the requirements required by that particular course they automatically get first preference."
So we are only considering applicants who meet the compulsory subjects.

How to check compulsory subjects?
- For a given course, the compulsory subjects are a list of subjects.
- We require that in at least one sitting, the applicant has taken all the compulsory subjects and passed (with at least a C? or any grade? The problem doesn't specify).
- Since the problem doesn't specify a minimum grade for compulsory subjects, we assume that having the subject in the sitting (with any grade) is enough?
But note: the points are only for A, B, C. So if a subject is not passed (i.e., below C) then it might not be considered?
However, the problem doesn't specify the passing grade. We assume that the applicant must have at least a C in the compulsory subjects?

Let's assume:
- To meet a compulsory subject, the applicant must have at least a C in that subject in one of the sittings.

But note: the problem says "they have all the requirements", so we assume the requirements include both the subjects and the grades?
Since the problem doesn't specify, we will assume that the requirement is just the subject, and the grade doesn't matter for meeting the requirement?
But then why are we calculating points? The points are used for ranking when there are more applicants than places.

We have to break ties by points. So we will calculate the total points for the applicant for the course.

How to calculate points?
- We are going to use the sitting with the lowest year (the first sitting) in which the applicant has

all the compulsory subjects (with at least C) and then calculate the total points for the 5 subjects in that sitting?

Alternatively, we might take the best sitting (the one with the highest total points) in which the applicant has the compulsory subjects?
But the problem says "First sitting is characterised by the lowest year", so we are to use the first sitting.

Let's assume:
- We are to use the first sitting (the one with the lowest year) in which the applicant has all the compulsory subjects (with at least C) and then use the grades in that sitting to calculate the points.

If an applicant doesn't have a sitting with all compulsory subjects, then they are not eligible for the course.

Steps for the algorithm:

For each course:
1. Filter applicants who meet the compulsory subjects (at least C in each compulsory subject in at least one sitting).
But note: we are using the first sitting (lowest year) in which they meet the compulsory subjects.

text

How to do this?

- For each applicant, sort their sittings by year (ascending).

- For each sitting in ascending order, check if the sitting has all the compulsory subjects with at least a C.

- If found, then this sitting is the one we use for the applicant for this course.

2. For each eligible applicant, calculate the total points for the 5 subjects in the sitting we found (using the point conversion for that sitting's year).

3. Now, we have a list of eligible applicants for the course, each with:

- applicant_id, disability_status, gender, total_points

4. Then, we do:

Step A: Disabled applicants who are eligible get first preference. We allocate them first until the course is full or we run out of disabled applicants.

Step B: After allocating disabled, we have remaining places = class_size - number of admitted disabled.

Step C: We then reserve 50% of the remaining places for females (rounded up? or down? The problem doesn't specify. Let's assume we round down).

Step D: We then split the remaining places into two pools:

- female quota: female_quota = floor(remaining_places * 0.5)
- male quota: male_quota = remaining_places - female_quota

Step E: We then rank the remaining applicants (non-disabled) by points (lower points are better? or worse? The problem doesn't specify).

Since the points are based on grades (A=1 is best, then B=2, etc.) and sitting (earlier sitting has lower points for the same grade), then lower points are better.

But note: the problem says "consideration is also by points". So we rank by points in ascending order (lower points are better).

Step F: We then select the top female_quota females from the non-disabled female applicants (by points, ascending) and the top male_quota males from the non-disabled male applicants (by points, ascending).

Step G: If the female quota is not filled, then the remaining places are given to the males (including those who were not in the top male_quota) by points (so we take the next best males until the female quota is filled or we run out of males).

Similarly, if the male quota is not filled, then the remaining places are given to the females.

However, the problem says: "If the number of females does not fill in all their quota, then the remaining number of places is given to males who would have failed to get places in the 50% quota of males and consideration is also by points."

This seems to say that if the female quota is not filled, then we take the males that were not allocated in the male quota (the ones that were below the male_quota cutoff) and then we consider them by points.

But note: the male quota was already allocated with the top male_quota males. So the remaining males are the ones that were not allocated. Then we take these remaining males and rank them by points and then fill the leftover from the female quota.

Similarly, if the male quota is not filled, then the leftover is given to the females that were not allocated.

However, the problem only specifies the case for females not filling their quota.

Let's break down:

- We have two groups: non-disabled females and non-disabled males.

- We first allocate the female_quota to the top females (by points, ascending).

- Then we allocate the male_quota to the top males (by points, ascending).

- Then, if there are still places left (because we didn't have enough females to fill the female_quota), then we take the remaining places and allocate them to the next best males (the ones that were not in the top male_quota, i.e., the next in the male ranking).

Similarly, if the male_quota is not filled, then we take the next best females.

But note: the problem says "50% is reserved for females" and then "If the number of females does not fill in all their quota, then the remaining number of places is given to males who would have failed to get places in the 50% quota of males".

This means that if we have leftover from the female quota, we give it to the males that were not allocated in the initial male quota (the ones that were below the male_quota cutoff).

We can do:

- Let total_remaining = class_size - number of disabled admitted.

- Let female_quota = floor(total_remaining * 0.5)

- Let male_quota = total_remaining - female_quota

- Then, we take the non-disabled females and sort them by points (ascending). We take the top female_quota females (if available) and mark them as admitted.

- Then, we take the non-disabled males and sort them by points (ascending). We take the top male_quota males (if available) and mark them as admitted.

- Then, if we have leftover places (because we didn't have enough females to fill the female_quota), then we take the next best males (the ones that were not in the top male_quota) until we fill the leftover or run out of males.

- Similarly, if we have leftover from the male quota (because we didn't have enough males to fill the male_quota), then we take the next best females.

However, note that the problem doesn't specify what happens if both quotas are not filled? We assume that we then just take the next best applicants regardless of gender? But the problem doesn't say.

Since the problem only specifies the case for females, we will assume that the leftover from one quota is given to the other gender.

But note: the problem says "50% is reserved for females" and then the leftover from females goes to males. It doesn't specify the reverse.

However, for symmetry, we can do the same for males: if the male quota is not filled, then the leftover goes to females.

But the problem doesn't say that. So we will stick to the problem statement:

- If female quota is not filled, then the leftover is given to males (the next best males that were not allocated).

- If the male quota is not filled, then we don't do anything? But then we would have empty seats.

This doesn't make sense. So let's assume that the 50% for females is a minimum reservation, but if we don't have enough males, then we can take more females.

Alternatively, we can interpret the problem as:

- We first set aside 50% of the remaining places (after disabled) for females, and 50% for males.

- Then, if one of the groups doesn't fill their quota, then the leftover is given to the other group.

This is a common way to handle quotas.

We'll do:

- Let total_remaining = class_size - number of disabled admitted.

- Let female_quota = ceil(total_remaining * 0.5) ? or floor? The problem doesn't specify.

Since the problem says "50% is reserved", we can take exactly 50% and if it's a fraction, we have to decide.

Let's assume we use integer division (floor) for the female quota, and then the male quota is total_remaining - female_quota.

Example: if total_remaining=5, then female_quota=2, male_quota=3.

Then, we allocate the top 2 females and top 3 males.

If we have only 1 female, then we allocate 1 female and 3 males, then we have 1 leftover? But wait, we allocated 1 female and 3 males, that's 4, but we have 5 places?

Actually, we have 5-2=3 remaining after disabled, then we split into 2 and 3. Then we allocate 1 female and 3 males, so we have 1 leftover place because we didn't fill the female quota (we only got 1 female but reserved 2). Then we give that leftover to the next best male?

But note: we already allocated the top 3 males. So we take the next male (the 4th) and allocate him.

So the steps are:

- Admit disabled.
- remaining_places = class_size - disabled_count
- female_quota = floor(remaining_places * 0.5)
- male_quota = remaining_places - female_quota

- Then, we admit the top female_quota females (if available) and top male_quota males (if available).

- Then, if we have leftover places (because we didn't have enough females to fill the female_quota), then we admit the next best males (after the top male_quota) until we fill the leftover.

- Similarly, if we have leftover from the male quota (because we didn't have enough males to fill the male_quota), then we admit the next best females.

But note: the problem only specifies the case for females. However, we will do both for symmetry.

Step H: We then combine the admitted disabled, admitted females, and admitted males to form the final list for the course.

We do this for each course.

However, note: an applicant can apply to multiple courses? The problem doesn't specify. We assume that each applicant applies to one course? Or we are allocating each applicant to the best course?

The problem doesn't specify. We assume that we are processing each course independently and an applicant can be admitted to multiple courses? Or we are assigning each applicant to at most one course?

Since the problem doesn't specify, we will assume that we are processing each course independently and an applicant can be admitted to multiple courses.

But note: in reality, an applicant can only take one course. So we might have to do a centralized matching? However, the problem doesn't specify.

We'll stick to independent courses.

Implementation:

We'll define:

text

courses: list of course objects (or dictionaries) with:

course_id, class_size, compulsory_subjects (list of strings)

applicants: list of applicant objects (or dictionaries) with:

applicant_id, disability_status (boolean), gender (string: "male" or "female"), sittings: list of sitting records.

Each sitting record: {year: int, subjects: list of (subject, grade)}

Steps for each course:

text

1. Precompute the eligible applicants for the course:

   For each applicant:

   - Sort sittings by year (ascending)

   - For each sitting in sorted order:

     - Check if the sitting has all the compulsory subjects (with at least grade C).

       We assume the sitting has exactly 5 subjects, but we don't require that the sitting has exactly the compulsory subjects?

       We require that the sitting has at least the compulsory subjects (so if the sitting has extra subjects, that's fine).

   - If we find a sitting, then the applicant is eligible and we use that sitting to calculate points.

   - If we don't find any sitting, then the applicant is not eligible.

2. Now, we have a list of eligible applicants, each with:

   applicant_id, disability_status, gender, total_points (calculated from the sitting we found)

3. We then split the eligible applicants into:

   - disabled applicants (disability_status is True)

   - non-disabled females

   - non-disabled males

4. Sort the disabled applicants by total_points (ascending) and admit them until we reach the class_size or we run out of disabled.

5. Then, we have remaining_places = class_size - number of admitted disabled.

6. Calculate female_quota = floor(remaining_places * 0.5)

male_quota = remaining_places - female_quota

7. Sort the non-disabled females by total_points (ascending) and take the top female_quota.

Sort the non-disabled males by total_points (ascending) and take the top male_quota.

8. Now, if we have leftover places because we didn't have enough females (i.e., we only got F females and F < female_quota), then we have leftover_female = female_quota - F.

Then, we take the next best males (the ones that were not in the top male_quota) and sort them by points (ascending) and take the top leftover_female.

Similarly, if we have leftover because we didn't have enough males (i.e., we only got M males and M < male_quota), then we have leftover_male = male_quota - M.

Then, we take the next best females (the ones that were not in the top female_quota) and sort them by points (ascending) and take the top leftover_male.

9. Then, the final admitted list for the course is:

admitted_disabled + admitted_females (from step 7 and 8) + admitted_males (from step 7 and 8)

However, note: step 8 might lead to more than remaining_places if we are not careful? Let me check:

text

We have remaining_places = female_quota + male_quota.

We admit F females and M males from step 7, so we have admitted F + M, which is <= remaining_places.

Then, if we have leftover_female, we admit leftover_female more males, so total becomes F + M + leftover_female = F + M + (female_quota - F) = M + female_quota.

But note: we initially had male_quota = remaining_places - female_quota, so M + female_quota = M + (remaining_places - male_quota) = remaining_places + (M - male_quota).

But M <= male_quota, so M - male_quota <= 0, so we are not exceeding remaining_places? Actually, we are:

We have:

remaining_places = female_quota + male_quota.

We admit:

F (<= female_quota) and M (<= male_quota) and then we admit (female_quota - F) more males.

So total admitted after step 8: F + M + (female_quota - F) = M + female_quota.

But we reserved male_quota for males, and we admitted M males and then (female_quota - F) more males, so total males = M + (female_quota - F).

And we admitted F females.

So total = F + M + (female_quota - F) = M + female_quota.

But note: remaining_places = female_quota + male_quota.

So we are exceeding remaining_places if M + female_quota > female_quota + male_quota? That is, if M > male_quota.

But M is at most male_quota (because we only took top male_quota males in step 7). So M <= male_quota, so M + female_quota <= male_quota + female_quota = remaining_places.

So we are safe.

Similarly, if we have leftover_male, then we admit (male_quota - M) more females, so total = F + M + (male_quota - M) = F + male_quota.

And F <= female_quota, so F + male_quota <= female_quota + male_quota = remaining_places.

So we are safe.

However, note: we might not have enough males to fill the leftover_female? Then we stop when we run out of males.

Similarly, we might not have enough females to fill the leftover_male.

So we have to take the minimum of (leftover_female, available_males) and (leftover_male, available_females).

Implementation:

text

We'll do:

admitted_disabled = ... (from step 4)

```
remaining_places = class_size - len(admitted_disabled)

non_disabled_females = [ ... ] (sorted by points, ascending)
non_disabled_males = [ ... ] (sorted by points, ascending)

# Step 7: initial allocation
admitted_females = non_disabled_females[:female_quota]
admitted_males = non_disabled_males[:male_quota]

# Step 8: handle leftover

# If we didn't get enough females, then we take more males
if len(admitted_females) < female_quota:
    leftover_female = female_quota - len(admitted_females)
    # Then we take the next best males (after the first male_quota) up to leftover_female
    additional_males = non_disabled_males[male_quota: male_quota + leftover_female]
    admitted_males.extend(additional_males)

# If we didn't get enough males, then we take more females
if len(admitted_males) < male_quota:
    leftover_male = male_quota - len(admitted_males)
    additional_females = non_disabled_females[female_quota: female_quota + leftover_male]
    admitted_females.extend(additional_females)
```

Then, the final admitted list for the course is admitted_disabled + admitted_females + admitted_males.

However, note: we are using the same list for non_disabled_females and non_disabled_males, and we are taking slices. We have to be cautious because we might run out of applicants.

This algorithm ensures that we fill the course up to the class_size, provided there are enough eligible applicants.

Let's code accordingly.

But note: the problem says "50% is reserved for females" and then the leftover from females goes to males. We are doing exactly that.

However, note: we are using floor for female_quota. This means that if remaining_places is odd, then female_quota is less than half and male_quota is more. Then, if we don't have enough females, we are giving the leftover to males, which might lead to more than 50% males.

Example: remaining_places=5, female_quota=2, male_quota=3.
If we have only 1 female, then we admit 1 female and 3 males from step7, then we have 1 leftover female quota, so we take 1 more male (the next best after the top3). So total: 1 female and 4 males.

This is in line with the problem: the female quota was 2, but we only had 1, so the leftover (1) goes to males.

Similarly, if we have 0 females, then we admit 0 females and 3 males from step7, then we have 2 leftover female quota, so we take 2 more males, total 5 males.

Now, if we don't have enough males, then we do the same for females.

Example: remaining_places=5, female_quota=2, male_quota=3.
If we have only 2 males, then we admit 2 males and 2 females from step7, then we have 1 leftover male quota, so we take 1 more female (the next best after the top2 females). So total: 3 females and 2 males.

This is symmetric.

We'll code accordingly.

However, note: the problem doesn't specify what happens if both quotas are not filled? We are handling both.

Let's code the algorithm accordingly.

We assume that the input data is in the following format:

courses = [
{

```
"course_id": "C1",
"class_size": 10,
"compulsory_subjects": ["Math", "Physics", "Chemistry"]
},
...
]

applicants = [
{
"applicant_id": "A1",
"disability_status": True,
"gender": "male",
"sittings": [
{
"year": 2020,
"subjects": [
("Math", "A"),
("Physics", "B"),
("Chemistry", "C"),
("Biology", "A"),
("English", "B")
]
},
{
"year": 2021,
"subjects": [
("Math", "A"),
("Physics", "A"),
("Chemistry", "A"),
("Biology", "A"),
("English", "A")
]
}
]
},
...
]
```

We'll write a function to calculate the points for a given sitting (year and grade).

Let's define a helper function:

```
def calculate_points(year, grade):
if year == 2020:
points_map = {'A':1, 'B':2, 'C':3}
elif year == 2021:
points_map = {'A':2, 'B':3, 'C':4}
elif year == 2022:
points_map = {'A':3, 'B':4, 'C':5}
elif year == 2023:
points_map = {'A':4, 'B':5, 'C':6}
elif year == 2024:
points_map = {'A':5, 'B':6, 'C':7}
else:
# If year is outside, we use the latest? But the problem says up to 2024.
# We'll assume the sitting year is between 2020 and 2024.
return None
```

text

```
    return points_map.get(grade, None)   # If grade is not A, B, C, then return None (but we assume only A, B, C)
```

Now, we write the main algorithm for one course:

```
def allocate_course(course, applicants):
compulsory_subjects = set(course['compulsory_subjects'])
eligible_applicants = []
```

text

```
    for applicant in applicants:
        # Sort sittings by year (ascending)
        sittings = sorted(applicant['sittings'], key=lambda x: x['year'])
        found_sitting = None
        for sitting in sittings:
            # Check if the sitting has all the compulsory subjects with at least C.
            # We assume the sitting has 5 subjects, but we only care about the compulsory ones.
```

```
        subjects_in_sitting = {subject for subject, grade in sitting['subjects']}
        if compulsory_subjects.issubset(subjects_in_sitting):
            # Now, check that for every compulsory subject, the grade is at least C?
            # We assume that having the subject in the sitting is enough, but the problem doesn't specify.
            # Alternatively, we might require at least C? Let's assume at least C.

            # Let's check the grades for the compulsory subjects in this sitting.
            grades = {subject: grade for subject, grade in sitting['subjects']}
            # Check if every compulsory subject has grade in ['A','B','C']?
            if all(grades[subject] in ['A','B','C'] for subject in compulsory_subjects):
                found_sitting = sitting
                break

    if found_sitting is not None:
        # Calculate total points for the 5 subjects in this sitting.
        total_points = 0
        for subject, grade in found_sitting['subjects']:
            points = calculate_points(found_sitting['year'], grade)
            if points is None:
                # This subject has an invalid grade, we skip the applicant? Or use 0? But the problem says only A, B, C.
                # We assume only A, B, C, so this should not happen.
                total_points = 0
                break
            total_points += points
        else:
```

```
        # All subjects have valid points
        eligible_applicants.append({
            'applicant_id': applicant['applicant_id'],
            'disability_status': applicant['disability_status'],
            'gender': applicant['gender'],
            'total_points': total_points
        })

# Now, we have eligible_applicants. Let's split them.

disabled_applicants = [app for app in eligible_applicants if app['disability_status']]
non_disabled_applicants = [app for app in eligible_applicants if not app['disability_status']]

non_disabled_females = [app for app in non_disabled_applicants if app['gender'] == 'female']
non_disabled_males = [app for app in non_disabled_applicants if app['gender'] == 'male']

# Sort by total_points (ascending)
disabled_applicants.sort(key=lambda x: x['total_points'])
non_disabled_females.sort(key=lambda x: x['total_points'])
non_disabled_males.sort(key=lambda x: x['total_points'])

# Step 4: admit disabled until class_size or run out
admitted_disabled = disabled_applicants[:course['class_size']]
remaining_places = course['class_size'] - len(admitted_disabled)

# If we have no remaining places, we return admitted_disabled and done.
```

```
if remaining_places == 0:
    return admitted_disabled, [], []

# Step 6: calculate quotas
female_quota = remaining_places // 2   # floor division
male_quota = remaining_places - female_quota

# Step 7: initial allocation for non-disabled
admitted_females = non_disabled_females[:female_quota]
admitted_males = non_disabled_males[:male_quota]

# Step 8: handle leftover

# If we didn't get enough females, then we take more males
if len(admitted_females) < female_quota:
    leftover_female = female_quota - len(admitted_females)
    # Take the next best males (after the first male_quota) up to leftover_female
    additional_males = non_disabled_males[male_quota: male_quota + leftover_female]
    admitted_males.extend(additional_males)

# If we didn't get enough males, then we take more females
if len(admitted_males) < male_quota:
    leftover_male = male_quota - len(admitted_males)
    additional_females = non_disabled_females[female_quota: female_quota + leftover_male]
    admitted_females.extend(additional_females)
```

return admitted_disabled, admitted_females, admitted_males

However, note: we are returning the lists of admitted applicants (each applicant is represented by their data, including applicant_id, disability_status, gender, total_points).

We can then combine them to get the final list for the course.

We assume that the course allocation is independent, so we run this for each course.

This algorithm is for one course.

We can then run it for every course.

Let's test with a small example.

Example course:
course_id: "C1", class_size: 2, compulsory_subjects: ["Math", "Physics"]

Example applicants:

text

Applicant1:

disability_status: True

gender: male

sittings: [

{year:2020, subjects: [("Math","A"), ("Physics","B"), ("Chemistry","C"), ("Biology","A"), ("English","B")]}

]

-> eligible, total_points = 1 (Math) + 2 (Physics) + ... = 1+2+3+1+2 = 9

Applicant2:

disability_status: False

gender: female

sittings: [

{year:2020, subjects: [("Math","A"), ("Physics","A"), ...] } -> same subjects, all A's -> total_points=1*5=5

]

Applicant3:

disability_status: False

gender: male

sittings: [

{year:2020, subjects: [("Math","A"), ("Physics","A"), ...] } -> total_points=5

]

Steps:

text

Eligible applicants: all three.

Disabled: [Applicant1] (points=9)

Non-disabled: [Applicant2 (5, female), Applicant3 (5, male)]

Step4: admit disabled (Applicant1). remaining_places=1.

female_quota = 1//2 = 0, male_quota=1.

Then, we admit top 0 females -> none.

Then, we admit top 1 male -> Applicant3.

Then, we check: we have 0 females (which is equal to female_quota of 0) and 1 male (which is equal to male_quota of 1). So no leftover.

So final admitted: Applicant1 and Applicant3.

But note: Applicant2 has lower points than Applicant1 and same as Applicant3, but she is female and we didn't reserve any female quota because remaining_places=1 and we used floor.

This seems to follow the algorithm.

However, note: the problem says "50% is reserved for females". In this case, we reserved 0% for females because of floor.

Maybe we should use ceiling for female_quota? Let's check the problem: it says "50% is reserved for females".

In the case of 1 remaining place, 50% of 1 is 0.5, which we round down to 0. So no female quota.

This might be acceptable, but note that the problem says "after subtraction on places for the disabled and 50% is reserved for females".

We are reserving 50% of the remaining, but if we round down, then in the case of an odd number, the female quota is less than 50%.

Alternatively, we can use:

text

```
female_quota = (remaining_places + 1) // 2   # This rounds up if remaining_places is odd.
```

Then, in the example above, female_quota=1, male_quota=0.

text

Then, we admit the top 1 female (Applicant2) and then we have no male quota.

Then, we check: we have 1 female (which is equal to female_quota of 1) and 0 males (equal to male_quota of 0). So no leftover.

So final admitted: Applicant1 and Applicant2.

This seems more in line with the 50% reservation.

Let's check the problem: it says "50% is reserved for females". So if we have 1 remaining place, then 50% of 1 is 0.5, which is not an integer.

The problem doesn't specify rounding. But note: the problem says "a quota for females is also reserved after subtraction on places for the disabled and 50% is reserved for females".

We can interpret 50% as half of the remaining places, and if it's a fraction, then we round up for females? Or round down?

In many quota systems, when reserving 50% for females, if the number is odd, then we round up to ensure at least 50%? Or we round down?

The problem doesn't specify. We'll assume that we round up for females to ensure at least 50%? But note: the problem says "50% is reserved", which might mean exactly 50% if possible, but if not, then at least 50%? Or at most?

Let's see the example: if we have 3 remaining places, then 50% is 1.5. If we round up, then female_quota=2, male_quota=1. Then we have 2 females and 1 male, which is 66% females.

Alternatively, if we round down, then female_quota=1, male_quota=2, which is 33% females.

The problem says "50% is reserved", so we want at least 50% for females? Or exactly?

The problem doesn't specify. We'll stick to the common practice of rounding up for females to ensure at least 50%? But note: the problem also says that if the number of females does not fill their quota, then the leftover goes to males.

So if we round up and we don't have enough females, then we give the leftover to males, so we might end up with less than 50% females.

Example: remaining_places=3, female_quota=2 (rounding up). If we have only 1 female, then we admit 1 female and 2 males (the male_quota was 1, but then we have 1 leftover from female quota, so we take 1 more male). So total: 1 female and 2 males (33% female).

Alternatively, if we round down, then female_quota=1, male_quota=2. If we have 1 female, then we admit 1 female and 2 males (33% female). Same.

But if we have 2 females, then with rounding up we admit 2 females and 1 male (66% female). With rounding down, we admit 1 female and 2 males (33% female).

So rounding up is more favorable to females.

Since the problem says "50% is reserved", we'll assume that we round up for females.

Let's change:

text

```
female_quota = (remaining_places + 1) // 2   # This gives the ceiling of remaining_places/2.
```

Then, male_quota = remaining_places - female_quota.

Now, let's re-run the example with 1 remaining place:

text

```
female_quota = 1, male_quota=0.
```

Then, we admit the top 1 female (if available) and then if we don't have enough females, we give the leftover to males.

So in the example above with 1 remaining place and 1 female (Applicant2) and 1 male (Applicant3), we admit the female.

This seems to be more in line with the 50% reservation.

We'll use this.

So we change step 6 to:

text

```
female_quota = (remaining_places + 1) // 2
male_quota = remaining_places - female_quota
```

Now, let's test with remaining_places=2:
female_quota = (2+1)//2 = 3//2 = 1
male_quota = 1

This is exactly 50% for each.

remaining_places=3:
female_quota = (3+1)//2 = 4//2 = 2
male_quota = 1

So we are reserving 2 out of 3 for females, which is more than 50%.

This is acceptable because the problem says "50% is reserved", meaning at least 50%.

We'll use this.

Final code for one course:

```
def allocate_course(course, applicants):
compulsory_subjects = set(course['compulsory_subjects'])
eligible_applicants = []

text

    for applicant in applicants:
        sittings = sorted(applicant['sittings'], key=lambda x: x['year'])
        found_sitting = None
        for sitting in sittings:
            subjects_in_sitting = {subject for subject, grade in sitting['subjects']}
            if compulsory_subjects.issubset(subjects_in_sitting):
                grades = {subject: grade for subject, grade in sitting['subjects']}
                if all(grades[subject] in ['A','B','C'] for subject in compulsory_subjects):
                    found_sitting = sitting
                    break

        if found_sitting is not None:
            total_points = 0
            for subject, grade in found_sitting['subjects']:
                points = calculate_points(found_sitting['year'], grade)
                if points is None:
                    # This subject has an invalid grade, we skip the applicant.
                    total_points = 0
                    break
                total_points += points
            else:
                eligible_applicants.append({
                    'applicant_id': applicant['applicant_id'],
```

```
            'disability_status': applicant['disability_status'],
            'gender': applicant['gender'],
            'total_points': total_points
        })

disabled_applicants = [app for app in eligible_applicants if app['disability_status']]
non_disabled_applicants = [app for app in eligible_applicants if not app['disability_status']]

non_disabled_females = [app for app in non_disabled_applicants if app['gender'] == 'female']
non_disabled_males = [app for app in non_disabled_applicants if app['gender'] == 'male']

disabled_applicants.sort(key=lambda x: x['total_points'])
non_disabled_females.sort(key=lambda x: x['total_points'])
non_disabled_males.sort(key=lambda x: x['total_points'])

admitted_disabled = disabled_applicants[:course['class_size']]
remaining_places = course['class_size'] - len(admitted_disabled)

if remaining_places == 0:
    return admitted_disabled, [], []

female_quota = (remaining_places + 1) // 2
male_quota = remaining_places - female_quota

admitted_females = non_disabled_females[:female_quota]
admitted_males = non_disabled_males[:male_quota]
```

```
    if len(admitted_females) < female_quota:
        leftover_female = female_quota - len(admitted_females)
        additional_males = non_disabled_males[male_quota: male_quota + leftover_female]
        admitted_males.extend(additional_males)

    if len(admitted_males) < male_quota:
        leftover_male = male_quota - len(admitted_males)
        additional_females = non_disabled_females[female_quota: female_quota + leftover_male]
        admitted_females.extend(additional_females)

    return admitted_disabled, admitted_females, admitted_males
```

We then run this for each course.

Note: This algorithm does not consider that an applicant might be admitted to multiple courses. We assume independence.

If we want to avoid duplicate admission of the same applicant to multiple courses, we would have to do a centralized matching, but the problem doesn't specify.

We'll leave it as independent courses.

This is the algorithm.

I'll design a comprehensive algorithm for course admission that considers disabilities, gender quotas, and multiple sitting points. Here's the solution:

**Algorithm: Course Admission System**

**Data Structures**

python

```
class Applicant:
    id: string
    disability_status: boolean
```

```
    gender: string  # 'male' or 'female'
    course_preferences: list
    subject_grades: dict  # {subject: (grade, sitting_year)}
    total_points: float

class Course:
    id: string
    class_size: integer
    compulsory_subjects: list
    total_subjects_required: integer = 5
```

**Main Algorithm**

python

```
def allocate_courses(applicants, courses):
    # Step 1: Calculate points for each applicant
    for applicant in applicants:
        applicant.total_points = calculate_total_points(applicant.subject_grades)

    # Step 2: Process each course independently
    for course in courses:
        allocate_course_positions(course, applicants)

    return generate_admission_list(applicants, courses)

def calculate_total_points(subject_grades):
    total = 0
    point_mapping = {
```

```
        2020: {'A': 1, 'B': 2, 'C': 3},
        2021: {'A': 2, 'B': 3, 'C': 4},
        2022: {'A': 3, 'B': 4, 'C': 5},
        2023: {'A': 4, 'B': 5, 'C': 6},
        2024: {'A': 5, 'B': 6, 'C': 7}
    }

    for subject, (grade, year) in subject_grades.items():
        total += point_mapping[year][grade]

    return total

def meets_course_requirements(applicant, course):
    # Check if applicant has all compulsory subjects
    for subject in course.compulsory_subjects:
        if subject not in applicant.subject_grades:
            return False

    # Check if applicant has exactly 5 subjects total
    if len(applicant.subject_grades) != course.total_subjects_required:
        return False

    return True

def allocate_course_positions(course, applicants):
    # Filter eligible applicants for this course
```

```
eligible_applicants = [
    app for app in applicants
    if meets_course_requirements(app, course) and
       course.id in app.course_preferences
]

# Sort by points (descending)
eligible_applicants.sort(key=lambda x: x.total_points, reverse=True)

# Separate applicants by category
disabled_applicants = [app for app in eligible_applicants if app.disability_status]
able_bodied_applicants = [app for app in eligible_applicants if not app.disability_status]

# Separate able-bodied by gender
able_bodied_males = [app for app in able_bodied_applicants if app.gender == 'male']
able_bodied_females = [app for app in able_bodied_applicants if app.gender == 'female']

# Step 1: Allocate to disabled applicants (first preference)
admitted = []
remaining_positions = course.class_size

for applicant in disabled_applicants:
    if remaining_positions > 0:
        admitted.append(applicant)
        remaining_positions -= 1
    else:
```

```
        break

    # Step 2: Calculate gender quotas from remaining positions
    if remaining_positions > 0:
        female_quota = remaining_positions // 2  # 50% for females
        male_quota = remaining_positions - female_quota

        # Allocate to females
        females_admitted = able_bodied_females[:female_quota]
        admitted.extend(females_admitted)
        remaining_female_slots = female_quota - len(females_admitted)

        # Allocate to males
        males_admitted = able_bodied_males[:male_quota]
        admitted.extend(males_admitted)
        remaining_male_slots = male_quota - len(males_admitted)

        # Step 3: Redistribute unfilled female quota
        if remaining_female_slots > 0:
            # Get remaining males who didn't get in initial male quota
            remaining_males = able_bodied_males[male_quota:]
            # Take best remaining males for the unfilled female slots
            additional_males = remaining_males[:remaining_female_slots]
            admitted.extend(additional_males)

        # Handle unfilled male quota (if any)
```

```
        if remaining_male_slots > 0:
            # Get remaining females who didn't get in initial female quota
            remaining_females = able_bodied_females[female_quota:]
            # Take best remaining females for the unfilled male slots
            additional_females = remaining_females[:remaining_male_slots]
            admitted.extend(additional_females)

    # Store admission results
    course.admitted_students = admitted

def generate_admission_list(applicants, courses):
    admission_list = {}

    for course in courses:
        admission_list[course.id] = {
            'admitted_students': [
                {
                    'id': student.id,
                    'disability_status': student.disability_status,
                    'gender': student.gender,
                    'total_points': student.total_points
                }
                for student in course.admitted_students
            ],
            'total_admitted': len(course.admitted_students)
        }
```

```
return admission_list
```